

25BECS141

LAB PROGRAM 1

CODE-

```
C p1.c > main()
1 //implementing basic array operations:insertion,deletion,searching,and traversal.
2 #include<stdio.h>
3 int main()
4 {
5     int arr[100],n,i,pos,value,choice,found;
6     printf("Enter number of elements: ");
7     scanf("%d",&n);
8     printf("Enter elements: \n");
9     for(i=0;i<n;i++){
10         scanf("%d",&arr[i]);
11     }
12     do{
13         printf("\n--MENU---\n");
14         printf("1.Traversal \n 2.Insertion \n 3.Deletion \n 4.Searching \n 5.Exit\n");
15         printf("Enter your choice: ");
16         scanf("%d",&choice);
17         switch(choice)
18         {
19             case 1: //Traversal
20                 printf("Array Elements are: ");
21                 for(i=0;i<n;i++){
22                     printf("%d ",arr[i]);
23                 }
24                 printf("\n");
25                 break;
26             case 2: //Insertion
27                 printf("Enter position and value: ");
28                 scanf("%d %d",&pos,&value);
29                 for(i=n;i>pos;i--){
30                     arr[i]=arr[i-1];
31                 }
32                 arr[pos]=value;
33                 n++;
34                 printf("Updated array: ");
35                 for(i=0;i<n;i++){
36                     printf("%d ",arr[i]);
37                 }
38                 printf("\n");
39                 break;
40             case 3: //Deletion
41                 printf("Enter position to delete: ");
42                 scanf("%d",&pos);
43                 for(i=pos-1;i<n-1;i++){
44                     arr[i]=arr[i+1];
45                 }
46                 n--;
47                 printf("Updated array: ");
48                 for(i=0;i<n;i++){
49                     printf("%d ",arr[i]);
50                 }
51                 printf("\n");
52                 break;
53             case 4: //searching
54                 printf("Enter element to search: ");
55                 scanf("%d",&value);
56                 found=0;
57                 for(i=0;i<n;i++){
58                     if(arr[i]==value){
59                         found=1;
60                         printf("Element found at index %d\n",i);
61                     }
62                 }
63                 if(found==0){
64                     printf("Element not found\n");
65                 }
66                 break;
67             case 5:
68                 printf("Exiting...\n");
69                 return 0;
70         }
71     } while(choice!=5);
72 }
```

output-

```
C p1.c > main()
36         arr[pos-1]=value;
37         n++;
38         printf("Updated array: ");
39         for(i=0;i<n;i++){
40             printf("%d ",arr[i]);
41         }
42         printf("\n");
43         break;
44     case 3: //Deletion
45         printf("Enter position to delete: ");
46         scanf("%d",&pos);
47         for(i=pos-1;i<n-1;i++){
48             arr[i]=arr[i+1];
49         }
50         n--;
51         printf("Updated array: ");
52         for(i=0;i<n;i++){
53             printf("%d ",arr[i]);
54         }
55         printf("\n");
56         break;
57     case 4: //searching
58         printf("Enter element to search: ");
59         scanf("%d",&value);
60         found=0;
61         for(i=0;i<n;i++){
62             if(arr[i]==value){
63                 found=1;
64                 printf("Element found at index %d\n",i);
65             }
66         }
67         if(found==0){
68             printf("Element not found\n");
69         }
70         break;
71     case 5:
72         printf("Exiting...\n");
73         return 0;
74 }
```

```
PS D:\DS PROGRAMS\LABPROGRAMS> gcc p1.c -o p1
PS D:\DS PROGRAMS\LABPROGRAMS> ./p1
Enter number of elements: 5
Enter elements:
4 9 8 6 2

--MENU--
1.Traversal
2.Insertion
3.Deletion
4.Searching
5.Exit
Enter your choice: 2
Enter position and value: 2 10
Updated array: 4 10 9 8 6 2

--MENU--
1.Traversal
2.Insertion
3.Deletion
4.Searching
5.Exit
Enter your choice: 5
Exiting...
PS D:\DS PROGRAMS\LABPROGRAMS>
```

```

69         found=0;
70         for(i=0;i<n;i++){
71             if(arr[i]==value){
72                 found=1;
73                 printf("Element found at position %d\n",i+1);
74                 break;
75             }
76         }
77     }
78     if(found==0){
79         printf("Element not found\n");
80     }
81     break;
82     case 5:
83         printf("Exiting...\n");
84         break;
85     default:
86         printf("Invalid choice! \n");
87     }
88 }while(choice !=5);
89 return 0;
90 }

```

LAB PROGRAM 2

CODE-

```

C p2.c > main()
1 //Represent a sparse matrix using triplet form and perform transpose operation
2 #include<stdio.h>
3
4 int main()
5 {
6     int matrix[10][10],triplet[50][3],transpose[50][3];
7     int i,j,m,n,k=1;
8
9     printf("Enter rows and columns :");
10    scanf("%d %d",&m,&n);
11
12    printf("Enter matrix elements : \n");
13    for(i=0;i<m;i++){
14        for(j=0;j<n;j++){
15            scanf("%d",&matrix[i][j]);
16        }
17        //convert to triplet form
18        triplet[0][0]=m;
19        triplet[0][1]=n;
20        triplet[0][2]=0;
21
22        for(i=0;i<m;i++){
23            for(j=0;j<n;j++){
24                if(matrix[i][j]!=0){
25                    triplet[k][0]=i;
26                    triplet[k][1]=j;
27                    triplet[k][2]=matrix[i][j];
28                    k++;
29                    triplet[0][2]++;
30                }
31            }
32        }
33        printf("\n Triplet Representation : \n");
34        for(i=0;i<k;i++){

```

```

        printf("\n Triplet Representation : \n");
        for(i=0;i<k;i++){
            printf("%d %d %d\n",triplet[i][0],triplet[i][1],triplet[i][2]);
        }
        //Transpose
        transpose[0][0]=triplet[0][1];
        transpose[0][1]=triplet[0][0];
        transpose[0][2]=triplet[0][2];
        k=1;
        for(i=0;i<n;i++){
            for(j=1;j<triplet[0][2];j++){
                if(triplet[j][1]==i){
                    transpose[k][0]=triplet[j][1];
                    transpose[k][1]=triplet[j][0];
                    transpose[k][2]=triplet[j][2];
                    k++;
                }
            }
        }
        printf("\n Transpose Triplet: \n");
        for(i=0;i<k;i++){
            printf("%d %d %d \n",transpose[i][0],transpose[i][1],transpose[i][2]);
        }
        return 0;
    }

```

OUTPUT

```
PS D:\DS PROGRAMS\LABPROGRAMS> gcc p2.c -o p2
PS D:\DS PROGRAMS\LABPROGRAMS> ./p2
Enter rows and columns :3 3
Enter matrix elements :
1 0 0
0 2 0
3 0 4

Triplet Representation :
3 3 4
0 0 1
1 1 2
2 0 3
2 2 4

Transpose Triplet:
3 3 4
0 0 1
0 2 3
1 1 2
2 2 4
PS D:\DS PROGRAMS\LABPROGRAMS> 
```

LAB PROGRAM 3A

CODE-

```
p3a.c > push(int)
3  #define MAX 5
4
5  int stack[MAX];
6  int top=-1;
7
8  //Push
9  void push(int value){
10     if(top==MAX-1){
11         printf("Stack Overflow\n");
12     }else{
13         top++;
14         stack[top]=value;
15         printf("Pushed %d\n",value);
16     }
17
18     //Pop
19     void pop(){
20         if (top== -1)
21             printf("Stack underflow");
22         else{
23             printf("Popped %d\n",stack[top]);
24             top--;
25         }
26     }
27     //Display
28     void display(){
29         if(top== -1)
30             printf("Stack is empty\n");
31         else{
32             printf("Stack elements:\n");
33             for(int i=top;i>=0;i--){
34                 printf("%d\n",stack[i]);
35             }
36         }
37     }
38     //Main
39     int main(){
40         push(10);
41         push(20);
42         push(30);
43         push(40);
44         pop(10);
45         display();
46     }
47 }
```

Output-

```
PS D:\DS PROGRAMS\LABPROGRAMS> gcc p3a.c -o p3a
PS D:\DS PROGRAMS\LABPROGRAMS> ./p3a
Pushed 10
Pushed 20
Pushed 30
Pushed 40
Popped 40
Stack elements:
30
20
10
PS D:\DS PROGRAMS\LABPROGRAMS> 
```

LAB PROGRAM 3B

CODE-

```
C 3b.c > push(int)
1 //stack implementation using linked list
2 #include<stdio.h>
3 #include<stdlib.h>
4
5 //Define node structure
6 struct Node{
7     int data;
8     struct Node*next;
9 };
10 struct Node*top=NULL; //Top of stack
11
12 //Push operation
13 void push(int value){
14     struct Node*newNode=(struct Node*)malloc(sizeof(struct Node));
15
16     if(newNode==NULL){
17         printf("Stack Overflow(Memory not available)\n");
18         return;
19     }
20     newNode->data=value;
21     newNode->next=top;
22     top=newNode;
23     printf("%d pushed to stack\n",value);
24 }
25
26 //Pop operation
27 void pop(){
28     if(top==NULL){
29         printf("Stack Underflow(Empty stack)\n");
30         return;
31     }
32
33     struct Node*temp=top;
34     printf("%d popped from stack \n",top->data);
```

```
35     top=top->next;
36     free(temp);
37 }
38
39 // Peek operation
40 void peek() {
41     if (top == NULL) {
42         printf("Stack is empty\n");
43     } else {
44         printf("Top element is: %d\n", top->data);
45     }
46 }
47
48 // Traverse operation
49 void display() {
50     if (top == NULL) {
51         printf("Stack is empty\n");
52         return;
53     }
54     struct Node* temp = top;
55     printf("Stack elements: ");
56     while (temp != NULL) {
57         printf("%d -> ", temp->data);
58         temp = temp->next;
59     }
60     printf("NULL\n");
61 }
62
63 int main() {
64     int choice, value;
65
```

```

66     while (1) {
67         printf("\n--- Stack Menu ---\n");
68         printf("1. Push\n");
69         printf("2. Pop\n");
70         printf("3. Peek\n");
71         printf("4. Display (Traverse)\n");
72         printf("5. Exit\n");
73         printf("Enter your choice: ");
74         scanf("%d", &choice);
75
76         switch (choice) {
77             case 1:
78                 printf("Enter value to push: ");
79                 scanf("%d", &value);
80                 push(value);
81                 break;
82             case 2:
83                 pop();
84                 break;
85             case 3:
86                 peek();
87                 break;
88             case 4:
89                 display();
90                 break;
91             case 5:
92                 printf("Exiting...\n");
93                 exit(0);
94             default:
95                 printf("Invalid choice! Please try again.\n");
96         }
97     }
98     return 0;
99 }

```

Output-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc 3b.c -o 3b
PS D:\DS PROGRAMS\LABPROGRAMS> ./3b

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 1
Enter value to push: 80
80 pushed to stack

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 1
Enter value to push: 20
20 pushed to stack

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 4
Stack elements: 20 -> 80 -> NULL

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 5
Exiting...
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 4

Code-

```
p4.c > (pop)
1 // Convert an infix expression to postfix using a stack
2 #include<stdio.h>
3 #include<stdlib.h>
4 #include<string.h>
5 #include<ctype.h>
6
7 #define MAX 100
8
9 char stack[MAX];
10 int top = -1;
11
12 void push(char c){
13     if(top >= MAX - 1){
14         printf("Stack Overflow\n");
15         return;
16     }
17     stack[++top] = c;
18 }
19
20 char pop(){
21     if(top < 0){
22         return '\0';
23     }
24     return stack[top--];
25 }
26
27 char peek(){
28     if(top < 0){
29         return '\0';
30     }
31     return stack[top];
32 }
33
34 int precedence(char op){
35     switch(op){
36         case '+':
37         case '-':
38             return 1;
39         case '*':
40         case '/':
41             return 2;
42         case '^':
43             return 3;
44         default:
```

```

45     }
46     return 0;
47 }
48
49 int isOperator(char c){
50     return (c=='+' || c=='-' || c=='*' || c=='/' || c=='^');
51 }
52
53 void infixToPostfix(char *infix, char *postfix){
54     int i = 0, j = 0;
55     char c;
56
57     while((c = infix[i++]) != '\0'){
58         // If operand, add to output
59         if(isalnum(c)){
60             postfix[j++] = c;
61         }
62         // If '(', push to stack
63         else if(c == '('){
64             push(c);
65         }
66         // If ')', pop until '('
67         else if(c == ')'){
68             while(top >= 0 && peek() != '('){
69                 postfix[j++] = pop();
70             }
71             pop(); // remove '('
72         }
73         // If operator
74         else if(isOperator(c)){ // FIXED typo
75             if(c == '^') // right associative
76                 while(top >= 0 && precedence(peek()) > precedence(c)){
77                     postfix[j++] = pop();
78                 }
79             else {
80                 while(top >= 0 && precedence(peek()) >= precedence(c)){ // FIXED typo
81                     postfix[j++] = pop();
82                 }
83             }
84             push(c);
85         }
86     }
87
88     // Pop remaining operators
89     while(top >= 0){
90         postfix[j++] = pop();
91     }
92     postfix[j] = '\0'; // IMPORTANT
93
94 }
95
96 int main(){
97     char infix[MAX], postfix[MAX];
98
99     printf("Enter an infix expression: ");
100     scanf("%s", infix);
101
102     infixToPostfix(infix, postfix);
103
104     printf("Postfix Expression: %s\n", postfix);
105     return 0;
106 }
```

Output-

```

> ▾ TERMINAL
● PS D:\DS PROGRAMS\LABPROGRAMS> gcc p4.c -o p4
● PS D:\DS PROGRAMS\LABPROGRAMS> ./p4
Enter an infix expression: (A+B)*C
Postfix Expression: AB+C*
○ PS D:\DS PROGRAMS\LABPROGRAMS> 
```


LAB PROGRAM 5

```
1 //Evaluate a postfix expression using a stack.
2
3 #include <stdio.h>
4 #include <ctype.h>
5
6 #define MAX 100
7
8 int stack[MAX];
9 int top = -1;
10
11 // Push operation
12 void push(int value)
13 {
14     top++;
15     stack[top] = value;
16 }
17
18 // Pop operation
19 int pop()
20 {
21     return stack[top--];
22 }
23
24 int main()
25 {
26     char postfix[MAX];
27     int i, op1, op2, result;
28
29     printf("Enter postfix expression: ");
30     scanf("%s", postfix);
31
32     for(i = 0; postfix[i] != '\0'; i++)
33     {
34         // If character is operand
```

```
35         if(isdigit(postfix[i]))
36         {
37             push(postfix[i] - '0');
38         }
39         // If character is operator
40         else
41         {
42             op2 = pop();
43             op1 = pop();
44
45             switch(postfix[i])
46             {
47                 case '+':
48                     result = op1 + op2;
49                     break;
50
51                 case '-':
52                     result = op1 - op2;
53                     break;
54
55                 case '*':
56                     result = op1 * op2;
57                     break;
58
59                 case '/':
60                     result = op1 / op2;
61                     break;
62
63                 case '%':
64                     result = op1 % op2;
65                     break;
66
67                 default:
68                     printf("Invalid operator");
69                     return 1;
70             }
71
72             push(result);
73         }
74     }
75
76     printf("Result = %d", pop());
77
78     return 0;
79 }
80
```

Code-

OUTPUT-

```
PS D:\DS PROGRAMS\LABPROGRAMS> gcc p5.c -o p5
PS D:\DS PROGRAMS\LABPROGRAMS> ./p5
Enter postfix expression: 23+
Result = 5
PS D:\DS PROGRAMS\LABPROGRAMS> 
```


LAB PROGRAM 6A

```

C p6a.c > enqueue(int)
1 //Implement queue operations using arrays and linked lists.
2
3
4 //Queue operations using arrays
5
6 #include <stdio.h>
7 #define MAX 5
8
9 int queue[MAX];
10 int front = -1, rear = -1;
11
12 // Insert element
13 void enqueue(int value)
14 {
15     if(rear == MAX - 1)
16     {
17         printf("Queue Overflow\n");
18     }
19     else
20     {
21         if(front == -1)
22             front = 0;
23
24         rear++;
25         queue[rear] = value;
26
27         printf("%d inserted\n", value);
28     }
29 }
30
31 // Delete element
32 void dequeue()
33 {
34     if(front == -1 || front > rear)
35     {
36         printf("Queue Underflow\n");
37     }
38     else
39     {
40         printf("%d deleted\n", queue[front]);
41         front++;
42     }
43 }
44
45 // Display queue

```

```

45 // Display queue
46 void display()
47 {
48     int i;
49
50     if(front == -1 || front > rear)
51     {
52         printf("Queue is Empty\n");
53     }
54     else
55     {
56         printf("Queue Elements:\n");
57
58         for(i = front; i <= rear; i++)
59         {
60             printf("%d ", queue[i]);
61         }
62     }
63
64     printf("\n");
65 }
66
67 int main()
68 {
69     enqueue(10);
70     enqueue(20);
71     enqueue(30);
72
73     display();
74
75     dequeue();
76
77     display();
78
79     return 0;
80 }
81

```

CODE-

```

● PS D:\DS PROGRAMS\LABPROGRAMS> gcc p6a.c -o p6a
● PS D:\DS PROGRAMS\LABPROGRAMS> ./p6a
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30
○ PS D:\DS PROGRAMS\LABPROGRAMS>

```

OUTPUT-

```

C p6b.c > enqueue(int)
1 //Queue operations using Linked List
2
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 // Node structure
7 struct Node
8 {
9     int data;
10    struct Node *next;
11 };
12
13 struct Node *front = NULL;
14 struct Node *rear = NULL;
15
16 // Insert element
17 void enqueue(int value)
18 {
19     struct Node *newNode;
20
21     newNode = (struct Node*)malloc(sizeof(struct Node));
22
23     newNode->data = value;
24     newNode->next = NULL;
25
26     if(rear == NULL)
27     {
28         front = rear = newNode;
29     }
30     else
31     {
32         rear->next = newNode;
33         rear = newNode;
34     }
35
36     printf("%d inserted\n", value);
37 }
38
39 // Delete element
40 void dequeue()
41 {
42     struct Node *temp;
43
44     if(front == NULL)
45     {

```

```

46         printf("Queue Underflow\n");
47     }
48     else
49     {
50         temp = front;
51
52         printf("%d deleted\n", front->data);
53
54         front = front->next;
55
56         if(front == NULL)
57             rear = NULL;
58
59         free(temp);
60     }
61 }
62
63 // Display queue
64 void display()
65 {
66     struct Node *temp;
67
68     if(front == NULL)
69     {
70         printf("Queue is Empty\n");
71     }
72     else
73     {
74         temp = front;
75
76         printf("Queue Elements:\n");
77
78         while(temp != NULL)
79         {
80             printf("%d ", temp->data);
81             temp = temp->next;
82         }
83
84         printf("\n");
85     }
86 }
87
88 int main()

```

```

88 int main()
89 {
90     enqueue(10);
91     enqueue(20);
92     enqueue(30);
93
94     display();
95
96     dequeue();
97
98     display();
99
100    return 0;
101 }

```

PROGRAM 6B-
CODE-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p6b.c -o p6b
PS D:\DS PROGRAMS\LABPROGRAMS> ./p6b
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30
PS D:\DS PROGRAMS\LABPROGRAMS>

```

OUTPUT-

```

C p9.c > ...
1 //Implement singly linked list operations:Insertion,deletion, and search.
2 #include<stdio.h>
3 #include<stdlib.h>
4
5 struct Node{
6     int data;
7     struct Node* next;
8 };
9
10 int search(struct Node*head,int target);
11
12 struct Node* createNode(int data){
13     struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
14     newNode->data=data;
15     newNode->next=NULL;
16     return newNode;
17 }
18
19 void insertAtBeginning(struct Node**head,int data){
20     struct Node* newNode= createNode(data);
21     newNode->next=*head;
22     *head=newNode;
23     return;
24 }
25
26 void insertAtEnd(struct Node**head,int data){
27     struct Node* newNode=createNode(data);
28     if(*head==NULL){
29         *head=newNode;
30         return;
31     }
32     struct Node* temp = *head;
33     while(temp->next!=NULL){
34         temp=temp->next;
35     }
36     temp->next=newNode;
37 }
38
39 void insertAtPosition(struct Node**head,int data,int position){
40     struct Node* newNode=createNode(data);
41     if(position==0){
42         newNode->next=*head;
43         *head=newNode;
44         return;
45     }

```

```

47     struct Node* temp=*head;
48     for(int i=0;temp!=NULL && i<position-1;i++){
49         temp=temp->next;
50     }
51
52     if(temp==NULL){
53         printf("Position out of range \n");
54         return;
55     }
56
57     newNode->next=temp->next;
58     temp->next=newNode;
59 }
60
61 void deleteFromBeginning(struct Node**head){
62     if(*head==NULL)return;
63     struct Node*temp=*head;
64     *head=(*head)->next;
65     free(temp);
66 }
67
68 void deleteFromEnd(struct Node**head){
69     if(*head==NULL)return;
70     if((*head)->next==NULL){
71         free(*head);
72         *head=NULL;
73         return;
74     }
75
76     struct Node* temp=*head;
77     while(temp->next->next !=NULL){
78         temp=temp->next;
79     }
80
81     free(temp->next);
82     temp->next=NULL;
83 }
84
85 void deleteFromPosition(struct Node**head,int position){
86     if(*head==NULL)return;
87
88     struct Node*temp=*head;
89
90     if(position==0){

```

PROGRAM 9

CODE-


```

91     *head=temp->next;
92     free(temp);
93     return;
94 }
95
96 for(int i=0;temp!=NULL && i<position-1;i++){
97     temp=temp->next;
98 }
99
100 if(temp==NULL||temp->next==NULL){
101     printf("Position out of range\n");
102     return;
103 }
104
105 struct Node* next=temp->next->next;
106 free(temp->next);
107 temp->next=next;
108 }
109
110 void traverse(struct Node*head){
111     struct Node*temp=head;
112     while(temp!=NULL){
113         printf("%d->",temp->data);
114         temp=temp->next;
115     }
116     printf("NULL\n");
117 }
118
119 int search(struct Node*head,int target){
120     struct Node*temp=head;
121     while(temp!=NULL){
122         if(temp->data==target)return 1;
123         temp=temp->next;
124     }
125     return 0;
126 }
127
128 int main(){
129     struct Node*head=NULL;
130
131     insertAtEnd(&head,10);
132     insertAtEnd(&head,20);
133     insertAtEnd(&head,30);

```

```

134     traverse(head);
135
136     insertAtBeginning(&head,5);
137     traverse(head);
138
139     insertAtPosition(&head,25,2);
140     traverse(head);
141
142     insertAtPosition(&head,25,2);
143     traverse(head);
144
145     deleteFromBeginning(&head);
146     traverse(head);
147
148     deleteFromEnd(&head);
149     traverse(head);
150
151     deleteFromPosition(&head,1);
152     traverse(head);
153
154     printf("List contains 20:%s \n",search(head,20)?"Yes":"No");
155     printf("List contains 40:%s\n",search(head,40)?"Yes":"No");
156
157     return 0;
158 }

```

OUTPUT-

```
PS D:\DS PROGRAMS\LABPROGRAMS> gcc p9.c -o p9
PS D:\DS PROGRAMS\LABPROGRAMS> ./p9
10->20->30->NULL
5->10->20->30->NULL
5->10->25->20->30->NULL
5->10->25->25->20->30->NULL
10->25->25->20->30->NULL
10->25->25->20->NULL
10->25->20->NULL
List contains 20:Yes
List contains 40:No
PS D:\DS PROGRAMS\LABPROGRAMS> 
```